Clínica San Patrás

Documentación de Arquitectura del Sistema

Asignatura

Programacion II

Integrantes

Maria Paulina Paez Guzman

Estephany Ruales Mazo

Institucion

Universidad Libre de Pereira

Semestre

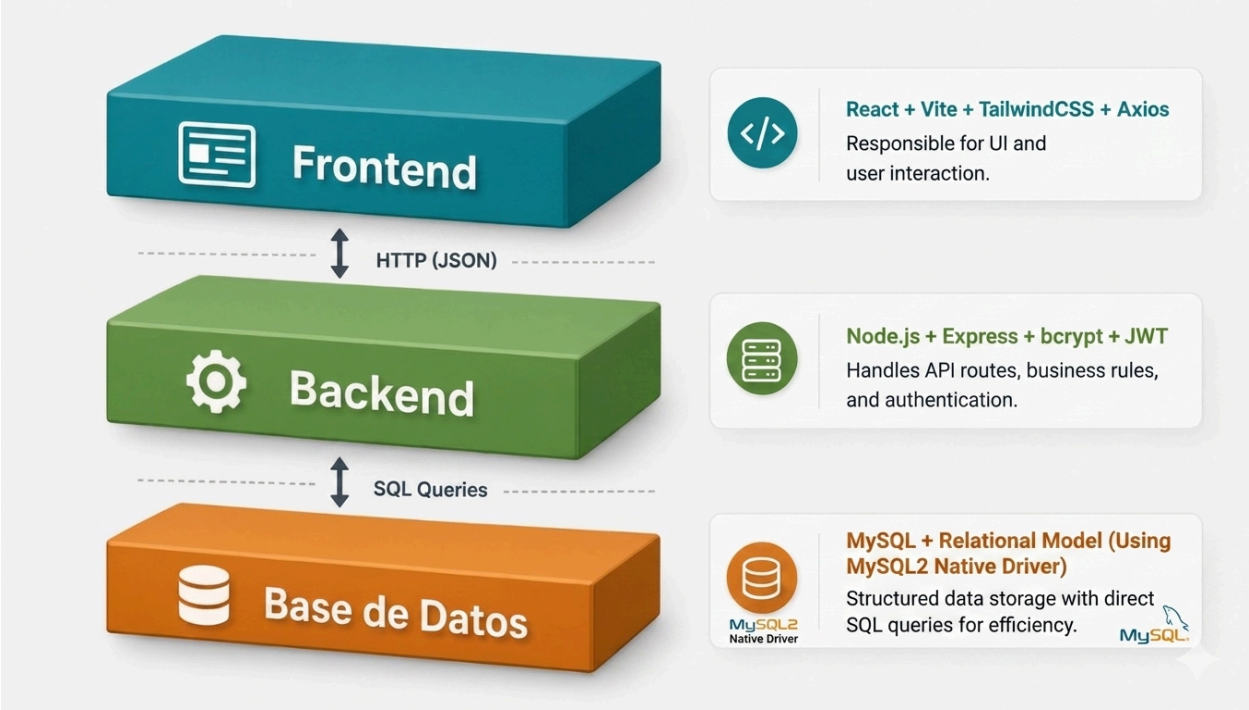
2026-1

Fecha

05 de Mayo de 2026

1. Diagrama de Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas, estándar de la industria para aplicaciones web empresariales. Cada capa tiene responsabilidades claras y se comunica únicamente con la capa adyacente, lo que garantiza separación de intereses, seguridad y facilidad de mantenimiento.



1.1 Flujo de una Petición Completa

A continuación se describe el ciclo de vida de una petición típica, por ejemplo: listar todos los pacientes.

#	Capa	Acción
1	**Usuario**	Hace clic en 'Ver Pacientes' en el menú lateral del Dashboard.
2	**Frontend**	React ejecuta: api.get('/pacientes') usando el cliente Axios centralizado.
3	**Frontend → Backend**	Axios agrega automáticamente el header: Authorization: Bearer <JWT>.

4	**Backend (middleware)**	verifyToken.js valida el JWT con jwt.verify(token, JWT_SECRET).
5	**Backend (controller)**	paciente.controller.js ejecuta: SELECT * FROM paciente ORDER BY apellidos.
6	**Base de Datos**	MySQL retorna el ResultSet con todos los registros de la tabla paciente.
7	**Backend**	Express responde: HTTP 200 + { success: true, data: [...], message: '...' }.
8	**Frontend**	Axios recibe la respuesta y React actualiza el estado: setPacientes(data).
9	**Usuario**	La tabla se re-renderiza mostrando los pacientes actualizados en pantalla.

2. Descripción de Componentes y Responsabilidades

2.1 Frontend — Capa de Presentación

Tecnologías: React 19 · Vite 8 · TailwindCSS 3 · Axios 1 · React Router 7 · SweetAlert2 11

Archivo / Componente	**Responsabilidad**
src/App.jsx	Punto de entrada de la aplicación. Configura React Router, define rutas públicas (/login) y protegidas (/dashboard, /pacientes, etc.). Contiene RutaProtegida y RutaAdmin para control de acceso en el frontend.
src/context/AuthContext.jsx	Contexto global de autenticación. Almacena { token, usuario, rol, imagen } en memoria y en localStorage. Expone las funciones login(), logout(), isAdmin() e isModerador() a todos los componentes.
src/services/api.js	Cliente Axios centralizado con la URL base de la API. Interceptor de petición: agrega el JWT automáticamente en Authorization: Bearer. Interceptor de respuesta: redirige al login si recibe HTTP 401.
src/services/authService.js	Funciones loginService() y registroService(). Encapsulan las llamadas a /api/auth/login y /api/auth/registro.

src/services/pacienteService.js	Funciones getPacientes(), createPaciente(), updatePaciente(), deletePaciente(). Cada función usa el cliente api.js.
src/services/medicoService.js	Funciones getMedicos(), createMedico(), updateMedico(), deleteMedico().
src/services/ingresoService.js	Funciones getIngresos(), createIngreso(), updateIngreso(), deleteIngreso(), estadisticasService().
src/pages/Login.jsx	Formulario controlado de login. Valida campos en el frontend, llama a loginService(), guarda la sesión en AuthContext y redirige al dashboard. Errores mostrados con SweetAlert2.
src/pages/Dashboard.jsx	Hub de navegación post-login. Muestra nombre, rol del usuario y 3 cards con estadísticas en tiempo real (total de pacientes, médicos e ingresos consultadas al backend). Accesos rápidos por rol.
src/pages/Pacientes.jsx	CRUD completo de pacientes: tabla con búsqueda, modal de creación/edición, confirmaciones SweetAlert2 en crear, editar y eliminar. Solo admin puede eliminar.
src/pages/Medicos.jsx	CRUD de médicos. Solo admin y moderador pueden crear/editar. Solo admin puede eliminar.
src/pages/Ingresos.jsx	CRUD de ingresos. Selección de paciente y médico mediante dropdowns cargados dinámicamente.
src/pages/Usuarios.jsx	Gestión de usuarios del sistema. Solo accesible para rol admin. Permite crear usuarios con bcrypt en el backend.
src/components/Layout.jsx	Layout compartido con sidebar de navegación, perfil del usuario, menú filtrado por rol y botón de logout con confirmación SweetAlert2.

2.2 Backend — Capa de Lógica de Negocio

Tecnologías: Node.js 18 · Express 5 · bcrypt 6 · jsonwebtoken 9 · mysql2 3 · cors · dotenv

Archivo	Responsabilidad
server.js	Punto de entrada del servidor. Inicializa Express, configura CORS (solo acepta http://localhost:5173), parsea JSON y monta todas las rutas bajo /api.
config/db.js	Pool de conexiones MySQL usando mysql2/promise. Lee credenciales desde .env. Limita a 10 conexiones simultáneas (connectionLimit: 10).
middleware/verifyToken.js	Extrae el JWT del header Authorization, lo verifica con jwt.verify(token, JWT_SECRET). Si el token es válido, adjunta req.user = { id, rol } para uso posterior. Devuelve HTTP 401 si falla.
middleware/checkRole.js	Middleware de fábrica: recibe una lista de roles permitidos y retorna un middleware. Verifica req.user.rol contra la lista. Devuelve HTTP 403 si el rol no tiene permiso.
controllers/auth.controller.js	login(): compara contraseña con bcrypt.compare(), genera JWT con expiración 8h. registro(): hashea con bcrypt.hash(password, 10) antes de insertar. perfil(): retorna datos del usuario autenticado. listarUsuarios(): solo admin.
controllers/paciente.controller.js	listar() con búsqueda LIKE. obtener() por ID. crear() con validaciones. actualizar(). eliminar() verifica FK antes de borrar (evita huérfanos).
controllers/medico.controller.js	Mismo patrón que paciente. Especialidad obligatoria. Verifica ingresos asociados antes de eliminar.
controllers/ingreso.controller.js	Incluye JOIN con paciente y médico para mostrar nombres. estadisticas() retorna COUNT de las 3 tablas principales para el dashboard.
routes/auth.routes.js	POST /login (público), POST /registro (admin), GET /perfil (autenticado), GET /usuarios (admin).

routes/paciente.routes.js	GET / (todos), GET /:id, POST / (autenticado), PUT /:id (autenticado), DELETE /:id (admin).
routes/medico.routes.js	GET / (todos), GET /:id, POST / (admin/mod), PUT /:id (admin/mod), DELETE /:id (admin).
routes/ingreso.routes.js	GET /estadisticas (antes de /:id para evitar conflicto), GET /, GET /:id, POST /, PUT /:id, DELETE /:id (admin).
.env	Variables sensibles: DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET, PORT. Nunca se sube al repositorio (incluido en .gitignore).
.env.example	Plantilla pública del .env con los nombres de variables pero sin valores reales. Permite a otros desarrolladores configurar el proyecto fácilmente.

2.3 Base de Datos — Capa de Persistencia

Base de datos: clinica_san_patras_db · Motor: MySQL 8 · Charset: utf8mb4_spanish_ci · Normalización: 3FN

Tabla	PK	Atributos relevantes
paciente	INT (PK auto)	nombre, apellidos (NOT NULL), direccion, poblacion, provincia, codigo_postal CHAR(5), telefono VARCHAR(15), fecha_nacimiento DATE (NOT NULL)
medico	INT (PK auto)	nombre, apellidos (NOT NULL), telefono VARCHAR(15), especialidad VARCHAR(80) (NOT NULL)
ingreso	INT (PK auto)	num_habitacion SMALLINT, cama TINYINT, fecha_ingreso DATETIME (DEFAULT NOW()), paciente_codigo INT (FK → paciente.codigo), medico_codigo INT (FK → medico.codigo)
usuarios	INT (PK auto)	usuario VARCHAR(50) UNIQUE, password VARCHAR(255) [hash bcrypt], imagen VARCHAR(255) NULL, rol ENUM('admin','usuario','moderador'), creado_en TIMESTAMP

Restricciones de integridad referencial:

- ingreso.paciente codigo → paciente.codigo (ON DELETE RESTRICT, ON UPDATE CASCADE)
- ingreso.medico código → médico.codigo (ON DELETE RESTRICT, ON UPDATE CASCADE)
- El RESTRICT impide eliminar un paciente o médico que tenga ingresos asociados, protegiendo la consistencia de los datos.

3. Lista de Endpoints de la API

Todos los endpoints responden con el formato estándar**:** success: boolean, data: any, message: string **.** La URL base es http://localhost:3001/api

3.1 Autenticación — /api/auth

Método	**URL**	**Auth**	**Rol requerido**	**Descripción / Body**
POST	/api/auth/login	No	—	Body: { usuario, password }. Respuesta: { token, usuario, rol, imagen, id }. HTTP 200 ok / 401 credenciales inválidas.
POST	/api/auth/registro	Sí	admin	Body: { usuario, password, rol, imagen }. Hashea contraseña con bcrypt 10 rounds. HTTP 201 / 409 usuario duplicado.
GET	/api/auth/perfil	Sí	Todos	Sin body. Retorna datos del usuario autenticado (sin password). HTTP 200 / 404.
GET	/api/auth/usuarios	Sí	admin	Sin body. Lista todos los usuarios (sin passwords). HTTP 200.

3.2 Pacientes — /api/pacientes

Método	**URL**	**Auth**	**Rol requerido**	**Descripción / Body**

GET	/api/pacientes	Sí	Todos	Query opcional: ?buscar=texto. Busca por nombre, apellidos o teléfono (LIKE). HTTP 200.
GET	/api/pacientes/:id	Sí	Todos	Sin body. Retorna un paciente por su código. HTTP 200 / 404.
POST	/api/pacientes	Sí	Todos	Body: { nombre*, apellidos*, direccion, poblacion, provincia, codigo_postal, telefono, fecha_nacimiento* }. HTTP 201.
PUT	/api/pacientes/:id	Sí	Todos	Body: mismo que POST. Actualiza todos los campos. HTTP 200 / 404.
DELETE	/api/pacientes/:id	Sí	admin	Sin body. Verifica que no tenga ingresos antes de eliminar. HTTP 200 / 409 conflicto FK.

3.3 Médicos — /api/médicos

Método	**URL**	**Auth**	**Rol requerido**	**Descripción / Body**
GET	/api/medicos	Sí	Todos	Query opcional: ?buscar=texto. Busca en nombre, apellidos, especialidad. HTTP 200.
GET	/api/medicos/:id	Sí	Todos	Sin body. Retorna un médico por código. HTTP 200 / 404.
POST	/api/medicos	Sí	admin/mod	Body: { nombre*, apellidos*, teléfono, especialidad* }. HTTP 201.
PUT	/api/medicos/:id	Sí	admin/mod	Body: mismo que POST. HTTP 200 / 404.

DELETE	/api/medicos/:id	Sí	admin	Sin body. Verifica ingresos asociados. HTTP 200 / 409.

3.4 Ingresos — /api/ingresos

Método	**URL**	**Auth**	**Rol requerido**	**Descripción / Body**
GET	/api/ingresos/estadísticas	Sí	Todos	Sin body. Retorna { total_pacientes, total_medicos, total_ingresos } para el dashboard. HTTP 200.
GET	/api/ingresos	Sí	Todos	Sin body. Lista todos los ingresos con JOIN: incluye nombre_paciente, nombre_medico, especialidad. HTTP 200.
GET	/api/ingresos/:id	Sí	Todos	Sin body. Retorna un ingreso con JOIN. HTTP 200 / 404.
POST	/api/ingresos	Sí	Todos	Body: { num_habitacion*, cama*, fecha_ingreso, paciente_codigo*, medico_codigo* }. HTTP 201 / 400 FK inválido.
PUT	/api/ingresos/:id	Sí	Todos	Body: mismo que POST. HTTP 200 / 404.
DELETE	/api/ingresos/:id	Sí	admin	Sin body. HTTP 200 / 404.

4. Justificación de Decisiones Técnicas

En esta sección se explican los fundamentos técnicos que sustentan la arquitectura y herramientas seleccionadas para el desarrollo de la aplicación.

4.1. Uso de JSON Web Tokens (JWT) para la Seguridad

Se seleccionó JWT como el mecanismo principal para la autenticación y autorización por las siguientes razones:

- Manejo de sesiones sin estado (Stateless): A diferencia de las sesiones tradicionales en el servidor, JWT permite que el servidor sea "stateless", ya que toda la información del usuario está contenida en el token firmado.
- Escalabilidad: Al no requerir almacenamiento de sesiones en la memoria del servidor o en una base de datos de sesiones, el sistema puede escalar horizontalmente con mayor facilidad.
- Seguridad y Portabilidad: El token se almacena de forma segura en el cliente (localStorage/sessionStorage) y se envía en cada petición mediante el encabezado Authorization: Bearer <token>, permitiendo un control de acceso granular basado en roles sin consultas constantes a la base de datos para validar la sesión.

4.2 Estructura de Carpetas en el Backend (Modularización)

El backend fue organizado siguiendo el patrón de diseño de separación de responsabilidades, dividiendo el código en carpetas como controllers, routes, middleware y models:

- Facilidad de Mantenimiento: Al separar la definición de las rutas (routes) de la lógica de negocio (controllers), el código se vuelve mucho más legible y fácil de depurar.
- Reutilización de Lógica (Middleware): El uso de la carpeta middleware permite centralizar funciones comunes, como la verificación de tokens o la validación de roles, y aplicarlas a múltiples rutas de forma sencilla, evitando la duplicación de código.
- Escalabilidad del Proyecto: Esta estructura permite que el proyecto crezca de manera organizada. Si se decide agregar una nueva entidad, simplemente se añaden sus archivos correspondientes en cada carpeta sin afectar el flujo actual del sistema.

4.3. Implementación de Variables de Entorno (.env)

Se implementó el uso del paquete dotenv para gestionar la configuración sensible del sistema a través de archivos .env:

- Protección de Datos Sensibles: Permite mantener fuera del código fuente información crítica como credenciales de la base de datos (usuario, contraseña) y la clave secreta del JWT (JWT_SECRET).
- Seguridad en el Control de Versiones: El archivo .env se incluye en el .gitignore para evitar que sea subido al repositorio de Git, cumpliendo con las mejores prácticas de seguridad informática.

- Portabilidad entre Entornos: Facilita el despliegue del sistema en diferentes entornos (Desarrollo, Pruebas o Producción) sin necesidad de modificar el código fuente, simplemente ajustando los valores en el archivo de entorno del equipo host.

4.4 bcrypt para Hash de Contraseñas

Las contraseñas nunca se almacenan en texto plano. bcrypt es el algoritmo estándar de la industria para hashear contraseñas porque incorpora un factor de trabajo (cost factor) configurable y un salt aleatorio generado automáticamente en cada hash, lo que hace imposible los ataques de tablas de arco iris (rainbow tables).

Se usan 10 rounds (10 iteraciones de la función hash), el mínimo recomendado por el docente. Con 10 rounds, hashear una contraseña tarda aproximadamente 100ms, lo suficientemente lento para dificultar ataques de fuerza bruta, pero suficientemente rápido para una buena experiencia de usuario en el login.

- bcrypt.hash(password, 10): genera hash + salt aleatorio + aplica 2^10 = 1024 iteraciones.
- bcrypt.compare(password, hash): verifica sin necesidad de conocer el salt (está embebido en el hash).
- La columna password en MySQL es VARCHAR(255) para alojar el hash completo de bcrypt (60 caracteres).

4.5 Elección de MySQL2 frente a Sequelize (ORM)

Se optó por utilizar el driver nativo mysql2 en lugar de un ORM como Sequelize por las siguientes razones fundamentales:

- Rendimiento y Menor Sobrecarga (Overhead): MySQL2 ejecuta consultas SQL de forma directa sin la capa de abstracción que requiere un ORM. Esto reduce el consumo de memoria y CPU en el servidor, permitiendo una respuesta más rápida en las consultas de ingresos y pacientes. Se evita la "traducción" constante de objetos JavaScript a sentencias SQL, lo cual es crítico en aplicaciones que buscan eficiencia pura.

- Control Total sobre SQL Nativo: El uso de MySQL2 permite escribir consultas complejas utilizando JOINs de forma natural (como en el historial de ingresos), optimizando la recuperación de datos en una sola operación sin depender de la lógica de carga (eager/lazy loading) propia de los ORMs. Permite aprovechar al máximo las capacidades específicas del motor MySQL sin las limitaciones que a veces imponen las abstracciones de nivel superior.

- Uso de Promesas y Async/Await: A pesar de ser un driver de bajo nivel, se utilizó la versión mysql2/promise. Esto permitió mantener un código moderno, limpio y fácil de

leer mediante el uso de asincronía, integrándose perfectamente con el flujo de trabajo de Node.js y Express utilizado en el proyecto.

4.6 SweetAlert 2 para Confirmaciones

El docente prohíbe explícitamente el uso de alert() nativo y window.confirm() del navegador. SweetAlert 2 reemplaza estas funciones con modales personalizables, accesibles y con soporte completo para Promesas (async/await), lo que permite esperar la decisión del usuario antes de ejecutar operaciones destructivas.

- Todas las operaciones de creación, edición y eliminación muestran un SweetAlert2 de confirmación antes de ejecutarse.
- Los errores del backend se muestran como SweetAlert2 de tipo 'error', con el mensaje devuelto por la API.
- El logout también requiere confirmación con SweetAlert2 para evitar cierres de sesión accidentales.

4.7 Control de Acceso Basado en Roles (RBAC)

El sistema implementa RBAC en dos niveles: frontend y backend. Esta doble capa es fundamental porque confiar solo en el frontend sería inseguro — cualquier usuario podría modificar el código del navegador.

Rol	**Permisos**
admin	Acceso total: CRUD completo de pacientes, médicos, ingresos y usuarios. Única opción de eliminar registros.
moderador	Puede crear y editar médicos y pacientes. No puede eliminar ni gestionar usuarios.
usuario	Solo puede ver y crear pacientes e ingresos. No puede modificar ni eliminar datos de médicos.

- Backend: checkRole() verifica el rol en req.user.rol en los endpoints sensibles. HTTP 403 si no tiene permiso.
- Frontend: el menú lateral solo muestra las opciones permitidas para el rol. Los botones de eliminar solo son visibles para admins.
- RutaAdmin en App.jsx redirige al dashboard si un no-admin intenta acceder a /usuarios directamente por URL.

5. Diagramas utilizados para el diseño de bases de datos

Diagrama relacional

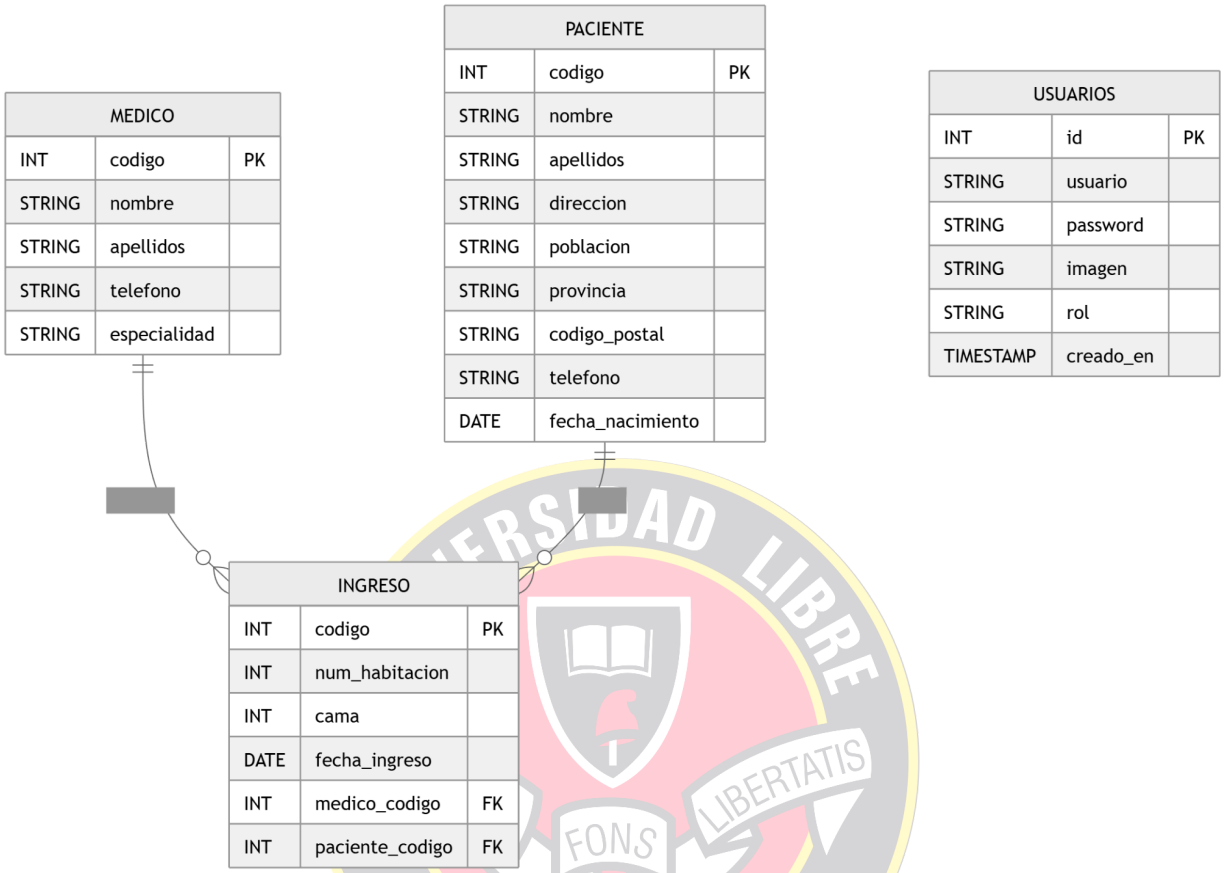
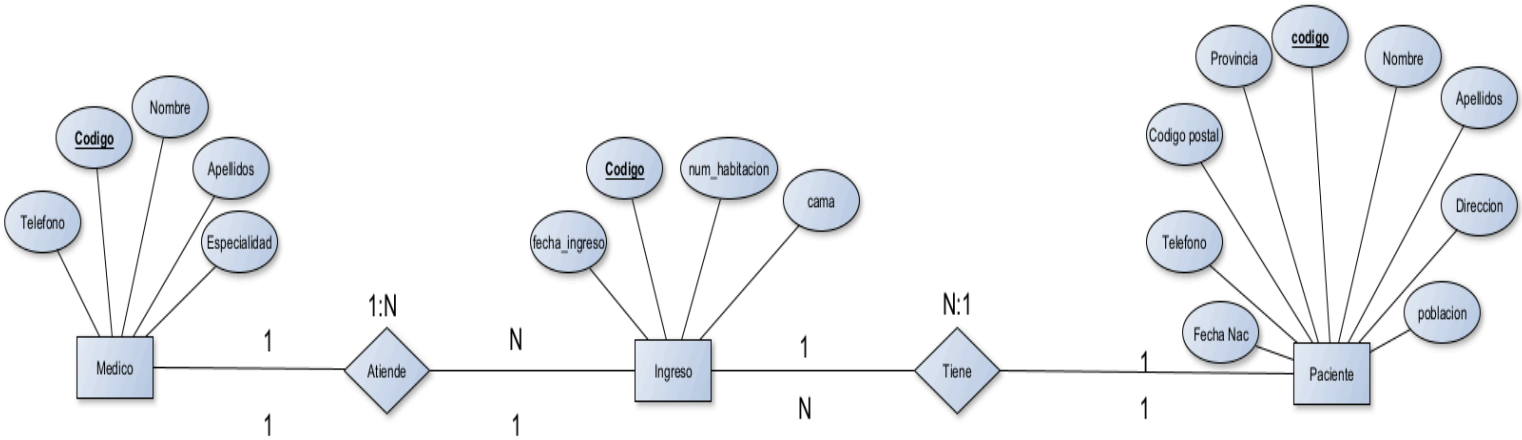



Diagrama entidad relación